

Chapter 2 | Exercise Answers

Database System Concepts and Architecture

Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, 7th edition.

Questions 2.1-2.15 follow the photographed chapter supplied for this course. Use the question numbers and printed page locators to read these answers beside your textbook.

These are original worked answers, not an official publisher solution manual. Open-ended designs state their assumptions. Full question text and photographed textbook figures are not reproduced.

Examples are identified as textbook data or original test data. SQL output tables are produced by executing the shown queries. SQLite-specific limitations are stated where they affect the result.

Publication of these answers does not expand the assigned sections or examination scope. Only taught and assigned material is assessed.

2.1. Terms and their roles

Source: photographed textbook, printed p./pp. 85; explanations 62-81.

Term	Meaning / example
Data model	Concepts for describing data, relationships, constraints and operations; for example, relations and keys.
Database schema	The relatively stable description of the database, not its current rows.
Database state	The data stored at a particular moment.
Internal schema	How records, files and access paths are represented in storage.
Conceptual schema	The community-wide logical structure and constraints.
External schema	The part and representation of data needed by a user group.
Data independence	Changing a schema at one level without requiring changes at the next higher level.
DDL	Data definition language: declares schemas and constraints.
DML	Data manipulation language: retrieves or changes stored data.
SDL	Storage definition language: specifies the internal schema.
VDL	View definition language: specifies external schemas and their mappings.
Query language	A high-level language for retrieving data.
Host language	A general-purpose language, such as Python, containing database calls.
Data sublanguage	The database-specific part embedded in a host language.
Database utility	A supporting program, such as a loader or backup utility.
Catalog	Stored descriptions of schemas, constraints and other metadata.
Client/server architecture	Clients request services from server components.
Three-tier architecture	Client interface, application server and database server are separated.
N-tier architecture	More than three functional tiers; for example, interface, gateway, business service and database.

2.2. Categories of data models

Source: photographed textbook, printed p./pp. 85; explanations 62-64, 80-83.

Category	What it describes	Example
Conceptual / high-level	Entities and relationships close to the users' meaning	ER / EER
Representational / logical	A structure implementable by a DBMS while hiding storage details	Relational, network, hierarchical
Physical / low-level	Record formats, placement and access paths	Files and indexes

Model	Organization	Useful contrast
Relational	Tables, tuples, attributes and keys	Relationships are represented through matching key values.
Object	Persistent objects with identity, attributes and behavior	Complex objects and inheritance can be represented directly.
XML	Hierarchical elements and attributes	Nested structure and optional elements suit document exchange; a schema may constrain them.

These categories describe abstraction, not a ranking. XML is self-describing at the element level; that does not make every document complete or valid.

2.3. Schema versus state

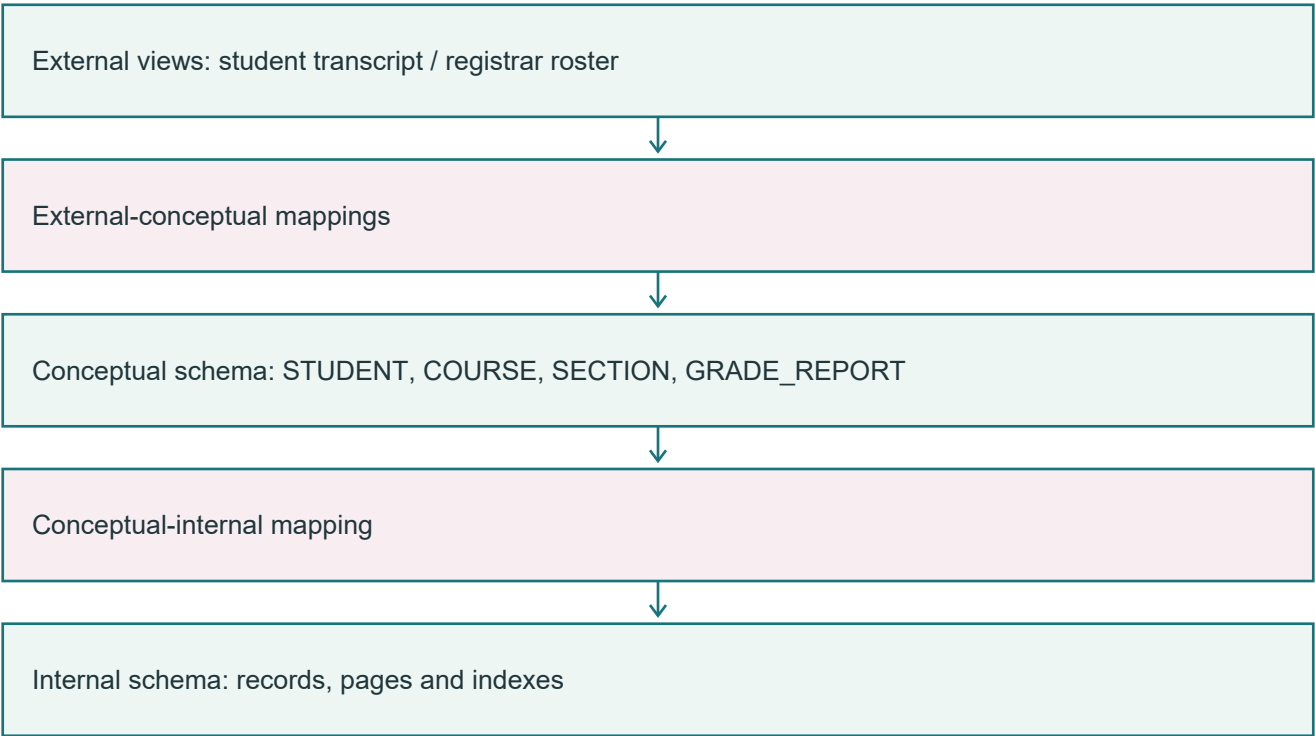
Source: photographed textbook, printed p./pp. 85; explanations 64-66.

Item	Before	After inserting one student
Schema	STUDENT(Student_number, Name, Major)	Unchanged
State	(17, Alex, MATH)	(17, Alex, MATH), (18, Blair, CS)

Adding a Phone attribute changes the schema. Adding a row changes the state. A valid state must satisfy the schema's constraints; a new row with Student_number 17 would violate a unique student-number rule.

2.4. Three schemas and mappings

Source: photographed textbook, printed p./pp. 85; explanations 66-68.



Mappings translate a request toward storage and reconstruct results for the requesting view. A student may see a transcript without seeing the storage layout or every other student.

Language	Role
DDL	Defines the conceptual schema.
SDL	Defines the internal schema.
VDL	Defines external schemas and their correspondence to the conceptual schema.

These are architectural roles. A practical DBMS can combine them into one language, such as SQL with CREATE TABLE and CREATE VIEW, rather than expose three separate languages.

2.5. Two kinds of data independence

Source: photographed textbook, printed p./pp. 85; explanations 67-68.

Kind	Change below	What should remain usable
Physical	Add an index or reorganize a file	The conceptual schema and application queries.
Logical	Split or extend conceptual tables while preserving a view	Existing external schemas and their applications.

Logical independence is usually harder: applications depend on the meaning and structure of data, so a structural change may need new view mappings. Physical changes can often be hidden entirely behind the same logical operations. Neither promises that every possible change is transparent.

2.6. Procedural and nonprocedural DML

Source: photographed textbook, printed p./pp. 85; explanations 68-70.

Approach	Example request	Who chooses the access sequence?
Procedural	Open the student file; visit records; keep those whose Major is CS.	The program specifies an operation sequence.
Nonprocedural	SELECT Name FROM STUDENT WHERE Major = 'CS';	The query states the desired rows; the DBMS chooses a plan.

A SQL query is declarative even when its physical plan uses a loop. SQL embedded in a procedural Python program does not make the query itself record-at-a-time DML.

2.7. Interfaces for different users

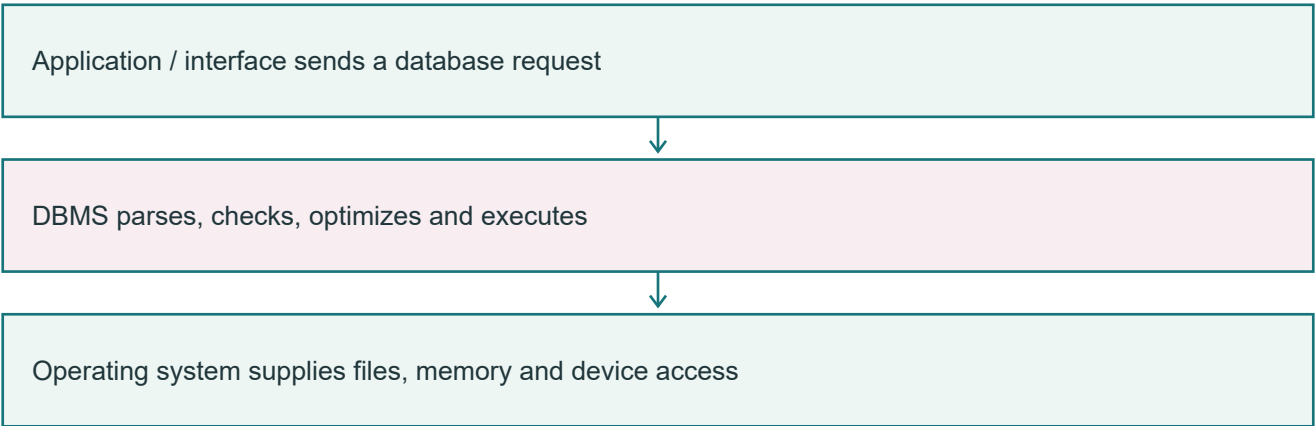
Source: photographed textbook, printed p./pp. 85; explanations 70-72.

Interface	Suitable user / task
Menus and mobile/web screens	Occasional users selecting known services.
Forms	Clerks entering a registration or updating an address.
Graphical interface	Users navigating diagrams or visual reports.
Natural language / keyword search	Casual users asking questions; meanings and access permissions still need checking.
Speech interface	Users who need hands-free or accessible interaction.
Parametric transaction screen	A teller or registrar repeatedly executes predefined operations.
DBA tools	Administrators define users, privileges, storage and maintenance.

An interface is appropriate when its permitted actions, validation and feedback match the user's job. A friendly interface does not remove the need for integrity constraints.

2.8. Software around a DBMS

Source: photographed textbook, printed p./pp. 85; explanations 72-76.

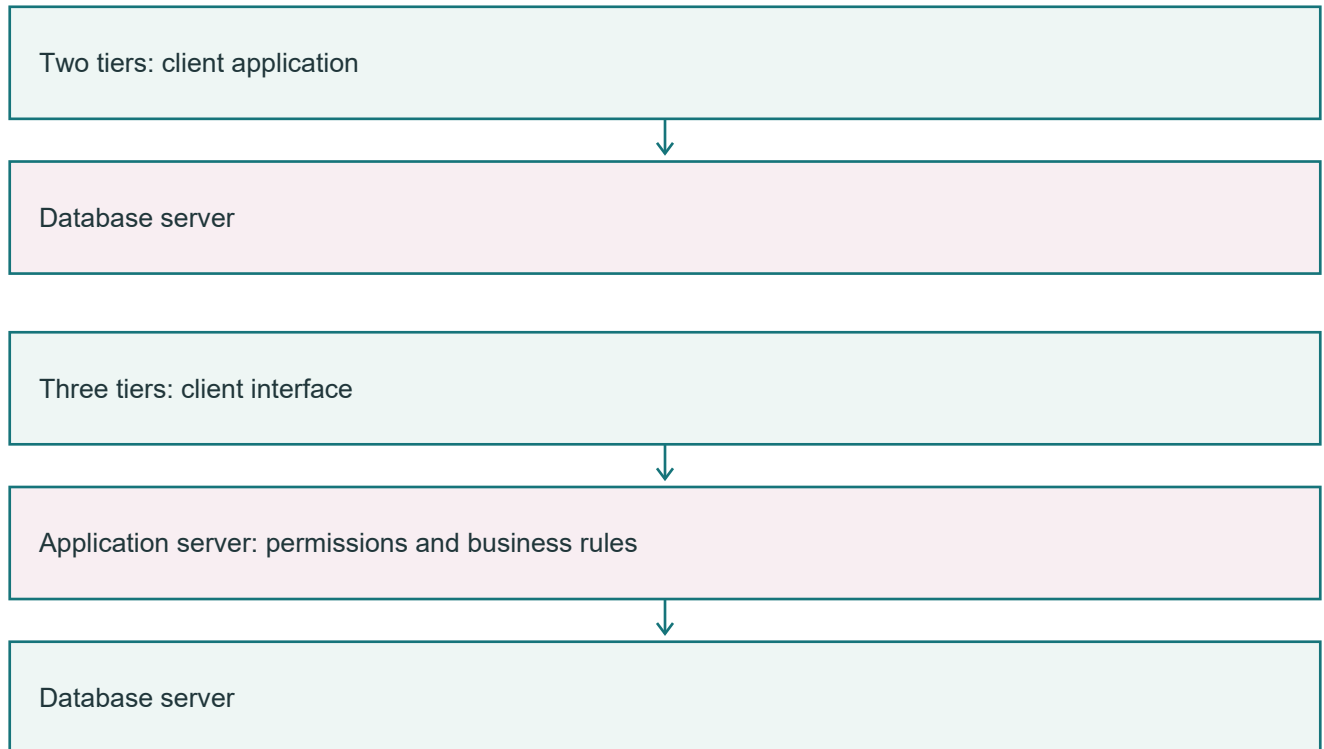


Other software	Interaction
Compiler / precompiler	Compiles host-language code and database calls.
Communication software	Carries requests and results between networked components.
Application server	Runs business rules and coordinates requests to the database.

The DBMS manages database semantics and recovery; the operating system does not automatically understand which rows form a valid business transaction.

2.9. Two tiers versus three tiers

Source: photographed textbook, printed p./pp. 85; explanations 77-80.



Two tiers let a client talk directly to the database service. Three tiers put application logic between the client and database. This centralizes rule updates and keeps database credentials off end-user clients, but adds another component to operate and secure.

2.10. Database utilities

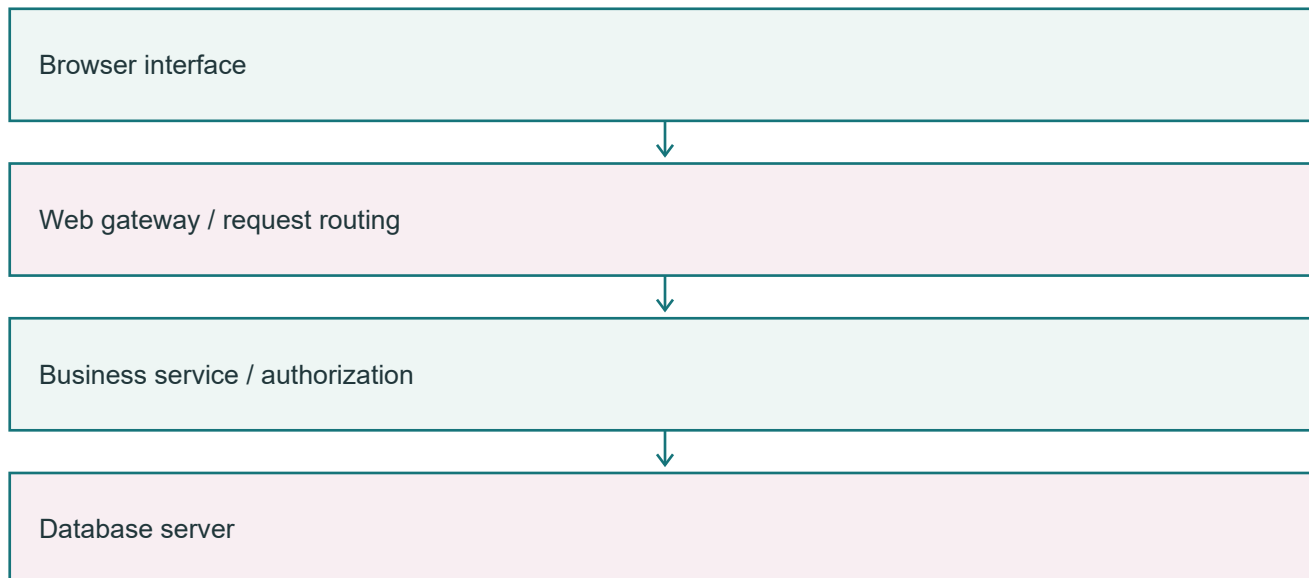
Source: photographed textbook, printed p./pp. 85; explanations 74-75.

Utility	Use
Loading	Convert incoming data into the database's representation; validate before accepting it.
Backup	Create recoverable copies; test restoration.
Storage reorganization	Rearrange records or access paths while preserving logical data.
Performance monitoring	Measure workloads and identify bottlenecks.
Design / CASE tools	Help create and maintain schemas and diagrams.
Data dictionary / catalog tools	Inspect metadata, definitions and dependencies.

A successful export is not proof that an operational database can be restored. Backup planning also needs recovery procedures and checks.

2.11. Why use more tiers?

Source: photographed textbook, printed p./pp. 85; explanations 79-80.



Additional tiers can separate presentation, routing, business rules and persistence so each can change or scale independently. A reporting service may be another tier.

The benefits must justify extra network calls, deployment work and failure handling. A tier is a functional separation; it does not necessarily require its own physical machine.

2.12. Users of the university database

Source: photographed textbook, printed p./pp. 85; Figure 2.1 p.64.

Figure 2.1 explicitly gives the schema for Figure 1.2. This answer uses that university schema; it does not invent unseen Figure 1.2 rows.

User	Application	Category / interface
Student	Read own sections, prerequisites and grades.	Casual end user; authenticated web interface.
Registrar clerk	Register a student and update course sections.	Parametric user; validated transaction form.
Instructor	Enter grades for assigned sections.	Parametric end user; roster form.
Department analyst	Compare enrollment by semester.	Sophisticated user; query/report tool.
DBA	Maintain access, backups and schema definitions.	Administrator; database administration tools.
Application programmer	Implement registration and transcript screens.	Developer; host language plus SQL.

The schema alone does not enforce who may read or edit a grade. Authorization must be added to the application and DBMS design.

2.13. An original database example

Source: photographed textbook, printed p./pp. 86; schema concepts 64-67.

Example: a small library records physical copies and individual borrowing events. Every copy has a unique CopyId. Repeated loans receive different LoanId values.

Relation	Key and attributes
MEMBER	MemberId (PK), Name
COPY	CopyId (PK), Title
LOAN	LoanId (PK), MemberId (FK), CopyId (FK), OutDate, ReturnedDate

LOAN.MemberId references MEMBER; LOAN.CopyId references COPY. These are relational references, not Chen ER relationships.

MEMBER: MemberId	Name
M1	Alex
M2	Blair

COPY: CopyId	Title
C1	Database Basics
C2	Database Basics

LOAN: LoanId	MemberId	CopyId	OutDate	ReturnedDate
L1	M1	C1	2026-09-01	2026-09-08
L2	M2	C1	2026-09-10	NULL

Constraints: references must exist; a return cannot precede its checkout; a copy has at most one open loan. The last rule needs a cross-row constraint or an equivalent transaction-safe implementation.

External view	Content
Member view	Only that member's loans and return status.
Circulation view	Copy availability and current borrower.
Management view	Loan counts by title, without member names.

Returning L2 changes the state; adding a renewal-limit attribute changes the schema.

2.14. Two DBMS examples

Source: photographed textbook, printed p./pp. 86; data models 62-64.

DBMS	Main model used here	Illustrative application
SQLite	Relational SQL tables	A local library checkout application.
MongoDB	Documents in collections	A product catalog whose products have different nested specifications.

SQLite can join a LOAN table to MEMBER. A MongoDB product document can embed a list of specifications. This illustrates representation, not a claim that either product is always faster or appropriate.

Primary product references: https://www.sqlite.org/lang_createtable.html and <https://www.mongodb.com/docs/manual/data-modeling/> . These examples were not benchmarked; no MongoDB instance was executed.

2.15. Choosing keys from stated rules

Source: photographed textbook, printed p./pp. 86.

Relation	Candidate key under the stated assumption	Why
Department_tbl	Department_Number	Each department is assigned a distinct department number. Address and name need not be unique.
Salaries_tbl	EmpNumber + From_date	Assume one salary period begins on a given date for a given employee. Different employees can share a start date.

EmpNumber	From date	Salary Amount
E1	2026-01-01	40000
E1	2026-07-01	45000
E2	2026-01-01	40000

EmpNumber alone cannot identify a salary-history row; From_date alone cannot either. If two records may start on the same date for the same employee, the question does not supply enough rules to establish this key. Add an explicit sequence or record identifier only after deciding that requirement.

Salary_Amount is not a key. To_date is a period boundary, not automatically a unique identifier. Enforce nonoverlapping periods separately if required.